5. Circle the rows where LED = 1. That's your "spec."

This is how real chip design begins: define behaviour across all cases – then implement.

Combinational logic

Combinational logic means output depends only on the current inputs – no memory, no history. You change an input, the output updates after a small delay.

Three blocks you'll see everywhere:

Multiplexer (MUX): "choose one input"

A MUX selects between signals. In phones, routers, and SoCs, MUXes are everywhere – choosing data paths, selecting sources, routing control signals.

Comparator: "is A bigger than B?"

Comparators decide which value wins – used in control logic, sensor thresholds, power regulation decisions, and more.

Adder: the core of "computation"

Even if you don't care about CPUs, adders show up in address calculations, counters, timers, PWM generators, and checksums.

Mental note: Combinational logic is your decision layer. It's what you write when you want "if this, then that" – but in hardware form.

Sequential logic

The moment you add memory, you enter sequential logic land. Sequential circuits depend on inputs *and* their previous state – usually implemented using flip-flops as memory elements.

The clock: the chip's traffic light

A clock is a repeating signal that tells flip-flops *when* to sample inputs and update outputs. Think of it as a citywide "now!" pulse. Between pulses, combinational logic settles. On the pulse, new state is captured.

Setup and hold time: the two rules that prevent chaos

Flip-flops need inputs to be stable around the clock edge:

- **Setup time:** input must be stable *before* the clock edge
- **Hold time:** input must remain stable *after* the clock edge

If you violate these, the flip-flop may capture the wrong value – or worse, go metastable.

Registers, counters, and timers

Once you get flip-flops, the rest becomes easy to follow:

Registers

A register stores a multi-bit value (like an 8-bit number). In design terms: registers hold state.

TRY THIS TONIGHT

For these workshops, you can do *everything* with just:

- Notebook + pen/pencil (truth tables, FSM diagrams, counter math)
- Calculator / phone calculator (powers of two, clock-to-time conversions)

Nice-to-have (makes it faster/cleaner, still optional):

- A laptop/PC with a simple digital-logic simulator like Logisim-evolution (to draw gates, MUXes, and quick circuit sketches instead of doodling).

Only if you want to physically "feel" the PWM/FSM/button ideas on real hardware (optional in Chapter 6):

- A beginner FPGA dev board with at least 1 LED + 1 button (many boards include these)
- USB cable to program it, plus a USB port on your PC
- *(Optional, for button debouncing as an electronics mini-build instead of a concept exercise):* breadboard, tactile pushbutton, resistors, capacitor, and a simple conditioning stage (common hardware debounce approaches are outlined well by DigiKey).

That's it, these workshops are designed so the minimum viable kit is literally pen + paper.

Counters

A counter increments every clock cycle (or on an enable). Counters become:

- timers ("wait 1 second"),
- PWM generators ("dim LED"),
- baud rate generators ("UART timing"),
- watchdogs ("if nothing happens for X, reset").

Finite State Machines

Most "behaviour" in electronics is an FSM: traffic lights, elevator logic, Wi-Fi connection states, USB negotiation, camera modes. An FSM has:

- a present state,
- inputs,
- next state logic,
- and outputs that depend on state (and sometimes inputs).

Two common styles:

- **Moore machine:** outputs depend only on current state
- **Mealy machine:** outputs depend on current state + inputs